TRABAJO PRACTICO 1

Historia, evolución y características de los lenguajes de programación

Ejercicio 1)

Los Lenguajes mas representativos son

- 1951 - 1955: Lenguajes tipo assembly
- 1956 - 1960: FORTRAN, ALGOL 58, ALGOL 60, LISP
- 1961 - 1965: COBOL, ALGOL 60, SNOBOL, JOVIAL
- 1966 - 1970: APL, FORTRAN 66, BASIC, PL/I, SIMULA 67, ALGOL-W
- 1971 - 1975: Pascal, C, Scheme, Prolog
- 1976 - 1980: Smalltalk, Ada, FORTRAN 77, ML
- 1981 - 1985: Smalltalk 80, Turbo Pascal, Postscript
- 1986 - 1990: FORTRAN 90, C++, SML
- 1991 - 1995: TCL, PERL, HTML
- 1996 - 2000: Java, Javascript, XML

Indique para cada uno de los períodos presentados cuales son las características nuevas que se incorporan y cuál de ellos la incorpora.

- 1951-1955: Uso de mnemotécnicos (ADD, STORE) para traducir a código máquina, específico por arquitectura. Assembly.
- 1956-1960: Lenguajes de alto nivel para cálculos numéricos y científicos. FORTRAN.
- 1961-1965: Portabilidad entre computadoras y orientación a negocios. COBOL.
- 1966-1970: Simplicidad y bajo uso de memoria en sistemas de tiempo compartido. BASIC.

- 1971-1975: Programación estructurada para aprendizaje (Pascal) y declarativa basada en reglas (Prolog). Pascal y Prolog.
- 1976-1980: Orientación a objetos y tipado fuerte para seguridad y fiabilidad. Ada.
- 1981-1985: Entorno de desarrollo integrado (IDE) con compilador. Turbo Pascal.
- 1986-1990: Extensión de C con orientación a objetos y abstracción. C++.
- 1991-1995: Manipulación de texto (Perl) y estructura para páginas web (HTML). Perl y HTML.
- 1996-2000: Portabilidad, concurrencia y orientación a objetos con mínimas dependencias. Java.

Ejercicio 2

Escriba brevemente la historia del lenguaje de programación que eligió en la encuesta u otro de su preferencia.

JavaScript fue creado en 1995 por **Brendan Eich** en solo 10 días mientras trabajaba en Netscape Communications. Originalmente se llamó *Mocha*, luego *LiveScript*, y finalmente JavaScript — nombre elegido en parte por marketing, aprovechando la popularidad de Java en ese momento, aunque los dos lenguajes no tienen relación real.

Su propósito inicial era simple: agregar interactividad básica a las páginas web en el navegador Netscape Navigator. En 1996 Microsoft lanzó su propia versión llamada *JScript* para Internet Explorer, lo que generó problemas de compatibilidad y obligó a estandarizar el lenguaje bajo el nombre **ECMAScript** (1997), gestionado por la organización ECMA International.

Durante años fue visto como un lenguaje "de segunda", usado solo para efectos visuales simples. Eso cambió drásticamente en **2005** con la popularización de **AJAX**, que permitió actualizar partes de una página sin recargarla, abriendo la puerta a aplicaciones web complejas como Gmail y Google Maps.

El gran salto llegó en **2009** con **Node.js**, creado por Ryan Dahl, que llevó JavaScript al servidor y lo convirtió en un lenguaje de propósito general. Ese mismo año se publicó ECMAScript 5, y en **2015** llegó **ES6** (ECMAScript 2015), la actualización más importante de su historia, con clases, arrow functions, promesas y módulos.

Hoy JavaScript es el lenguaje más usado del mundo según Stack Overflow, y es la base de ecosistemas enormes como React, Vue, Angular y TypeScript.

Ejercicio 3

¿Qué atributos debería tener un buen lenguaje de programación? Por ejemplo, ortogonalidad, expresividad, legibilidad, simplicidad, etc. De al menos un ejemplo de un lenguaje que cumple con las características citadas.

SIMPLICIDAD Y LEGIBILIDAD: Python, Ruby

- Poder producir programas fáciles de escribir y de leer.
- Resultar fáciles a la hora de aprenderlo o enseñarlo.
- La estructura subyacente del algoritmo y los datos que el programa representa deben quedar en manifiesto al inspeccionar el texto del programa.

CLARIDAD EN LOS BINDINGS: Rust y Python

- Definición del lenguaje.
- Implementación del lenguaje.
- En escritura del programa.
- Compilación.
- Cargado del programa.
- En ejecución.

CONFIABILIDAD: Ada o Rust

- Chequeo de tipos: cuanto antes se encuentren errores menos costosos resulta realizar los arreglos que se requieran.
- Manejo de excepciones: la habilidad para interceptar errores en tiempo de ejecución, tomar medidas correctivas y continuar.

SOPORTE: Javascript, Java, C#

- Debería ser accesible para cualquiera que quiera usarlo o instalarlo. Lo ideal sería que su compilador o intérprete sea de dominio público.
- Debería poder ser implementado en diferentes plataformas. Java lo garantiza mediante la JVM ("write once, run anywhere").

- Deberían existir diferentes medios para poder familiarizarse con el lenguaje: tutoriales, cursos, textos, etc.

ABSTRACCIÓN: Java, Python

- Capacidad de definir y usar estructuras u operaciones complicadas de manera que sea posible ignorar muchos de los detalles.
- Concepto clave para manejar la complejidad, abstracción de procesos y de datos.

ORTOGONALIDAD: Python

- Significa que un conjunto pequeño de constructores primitivos puede ser combinado en un número relativamente pequeño de formas a la hora de construir estructuras de control y datos. Cada combinación es legal y con sentido.

EXPRESIVIDAD: JavaScript (ES6+)

- Permite decir mucho con poco código sin sacrificar claridad.
- JavaScript moderno incorpora arrow functions, destructuring y spread operators que permiten expresar operaciones complejas de forma concisa.

EFICIENCIA: C, C++, Rust, Go

- Tiempo y espacio: capacidad de un programa para realizar una tarea de manera correcta consumiendo pocos recursos de memoria, espacio en disco, tiempo de ejecución, tráfico de red, etc.
- Esfuerzo humano: que mejore el rendimiento del programador.
- Optimizable: capacidad del lenguaje de venir optimizado para tareas específicas.

Ningún lenguaje cumple todos estos atributos perfectamente — siempre hay trade-offs. Go sacrifica expresividad por simplicidad. Haskell maximiza ortogonalidad pero sacrifica legibilidad para principiantes. Esas decisiones de diseño son justamente lo que define la identidad de cada lenguaje

Ejercicio 4

Tome uno o dos lenguajes de los que ud. Conozca y

- **Describa los tipos de expresiones que se pueden escribir en él/ellos**
- **Describa las facilidades provistas para la organización del programa**

- **Indique cuáles de los atributos del ejercicio anterior posee el/los lenguaje/s elegidos y cuáles no posee, justifique en cada caso.**

Lenguaje elegido: JavaScript

Tipos de expresiones

Expresiones aritméticas: permiten operar con números usando los operadores clásicos. JS tiene algunas particularidades como la coerción implícita de tipos, por ejemplo "5" + 3 devuelve "53" en vez de 8 porque convierte el número a string automáticamente.

```
let resultado = (10 + 5) * 2 / 3;
```

Expresiones relacionales: comparan valores y devuelven un booleano. JS distingue entre == (igualdad con coerción) y === (igualdad estricta, sin coerción), siendo esta última la recomendada.

```
5 === 5       // true
"5" == 5      // true  (coerción)
"5" === 5     // false (estricta)
```

Expresiones lógicas: combinan condiciones booleanas

```
(edad >= 18 && tieneDNI) || esEmancipado
```

Expresiones de asignación: JS permite asignación simple y compuesta.

```
let x = 10;
x += 5;    // x = 15
x++;       // x = 16
```

Expresiones condicionales (ternarias):

```
let mensaje = edad >= 18 ? "mayor" : "menor";
```

Expresiones de función (arrow functions): en JS las funciones son ciudadanos de primera clase, pueden asignarse a variables, pasarse como argumento y devolverse.

```
const doble = x => x * 2;
const numeros = [1, 2, 3].map(x => x * 2); // [2, 4, 6]
```

Expresiones con objetos y destructuring:

```
const { nombre, edad } = persona;
const copia = { ...persona, edad: 30 };
```

Facilidades para la organización del programa

Funciones: la unidad básica de organización. Permiten encapsular lógica reutilizable. JS soporta funciones de primera clase, closures y funciones de orden superior.

Objetos y clases: JS es orientado a objetos basado en prototipos, aunque desde ES6 incorporó la sintaxis de clases que resulta más familiar.

```
class Animal {
    constructor(nombre) {
        this.nombre = nombre;
    }
    hablar() {
        return `${this.nombre} hace un sonido`;
    }
}
```

Módulos (ES6): permiten dividir el código en archivos separados con importaciones y exportaciones explícitas, fundamental para proyectos grandes.

```
// animal.js
export class Animal { ... }

// main.js
import { Animal } from './animal.js';
```

Promesas y async/await: organizan el código asincrónico de forma legible, evitando el llamado "callback hell".

```
const datos = await fetch('https://api.ejemplo.com/datos');
```

Namespaces y closures: permiten encapsular variables y evitar colisiones en el scope global.

Atributos del ejercicio anterior

Simplicidad y legibilidad — parcialmente: JS moderno (ES6+) es bastante legible, pero arrastra inconsistencias históricas como la coerción implícita de tipos, el comportamiento variable de `this`, y la diferencia entre `null` y `undefined`. No es tan simple como Python para aprender desde cero.

Claridad en los bindings — parcialmente: las variables declaradas con `let` y `const` tienen binding claro en tiempo de escritura. Sin embargo, `var` tiene comportamiento de hoisting que genera confusión, vinculando la variable en un momento no siempre evidente para el programador.

Confiabilidad — baja en JS puro, alta en TypeScript: JS es de tipado dinámico y débil, lo que significa que los errores de tipo aparecen en tiempo de ejecución y no en compilación. No tiene manejo de excepciones tan robusto como Java o Rust. TypeScript, que compila a JS, soluciona gran parte de esto agregando tipado estático.

Soporte — muy alto: es el lenguaje más soportado del mundo. El intérprete está integrado en todos los navegadores, Node.js es gratuito y de código abierto, y existe una cantidad enorme de documentación, cursos y tutoriales disponibles. Corre en cualquier plataforma donde haya un navegador o Node instalado.

Abstracción — alta: soporta programación orientada a objetos con clases, herencia y polimorfismo, y también programación funcional con funciones de orden superior, lo que da dos modelos de abstracción distintos en el mismo lenguaje.

Ortogonalidad — media: las construcciones básicas se pueden combinar razonablemente bien, especialmente con funciones como ciudadanos de primera clase. Sin embargo, las inconsistencias del lenguaje (como el comportamiento de `this` según el contexto) rompen la ortogonalidad en varios casos.

Expresividad — muy alta: es uno de los puntos más fuertes de JS moderno. Con pocas líneas se pueden expresar operaciones complejas usando arrow functions, destructuring, spread, map/filter/reduce y async/await.

Eficiencia — media: no es el lenguaje más eficiente en tiempo de ejecución comparado con C o Rust, pero los motores modernos como V8 (Chrome/Node) incluyen compilación JIT que mejora mucho el rendimiento. Es eficiente en términos de esfuerzo humano: se puede prototipar muy rápido.

Ejercicio 5 Lenguajes ADA

Describa las características más relevantes de Ada, referida a:

- **Tipo de datos**
- **Tipos abstractos de datos-paquetes**
- **Estructuras de datos**
- **Manejo de excepciones**
- **manejo de concurrencia**

Tipos de datos: En Ada, cada variable debe declararse con un tipo específico, y este no puede cambiar durante la ejecución del programa. Algunos tipos de datos disponibles son:

- Enteros (Integer)
- Caracteres (Character)
- Booleanos (Boolean)
- Reales (Float)
- Punteros (Access)
- Arreglos (Array)
- Registros (Record)
- Variantes (Variant)
- Enumeraciones
- Tipos derivados

Tipos abstractos de datos - paquetes: Los tipos de datos abstractos en Ada los define la persona que programa y se crean combinando tipos existentes. Los paquetes permiten encapsular datos y procedimientos relacionados en un único módulo.

Un paquete en Ada consta de dos partes principales: la declaración en la especificación (archivo .ads) y la implementación en el cuerpo (archivo .adb).

Puede incluir variables privadas y constantes accesibles solo dentro del paquete.

Estructuras de datos:

- Arrays: pueden tener índices dinámicos o estáticos, con verificación de límites en tiempo de ejecución.
- Registros: similares a las estructuras en C, con campos y posibles discriminantes para variantes.
- Punteros (Access Types): gestionados de forma segura, evitando referencias nulas no deseadas.
- Colas, árboles, conjuntos y mapas: no son estructuras nativas del lenguaje, sino que se implementan mediante bibliotecas externas o definiciones propias.

Manejo de excepciones: Ada gestiona excepciones mediante bloques `begin-exception-when`, propios del lenguaje. Cuando se lanza una excepción con `raise`, el programa busca el bloque `when` correspondiente para manejarla. Si existe, se ejecuta el código del bloque. Si no hay un manejador en el bloque actual, la excepción se propaga automáticamente al bloque llamador. Si no se encuentra ningún manejador, el programa finaliza con un mensaje de error.

```
begin
    -- código
exception
    when Constraint_Error =>
        -- manejo de error de rango
    when others =>
        -- cualquier otra excepción
end;
```

En Ada, las excepciones se declaran en la especificación del paquete mediante la cláusula `exception`. Además de declararlas, se puede añadir información adicional, como una descripción o el motivo de su ocurrencia.

Manejo de concurrencia:

- En Ada, los procesos concurrentes se definen creando tareas (tasks), que son unidades de ejecución independientes y pueden ejecutarse simultáneamente con otras tareas.

- Se proporcionan mecanismos seguros de comunicación entre tareas, como los objetos protegidos (protected objects) y el mecanismo de rendezvous, en el que dos tareas se sincronizan en un punto de encuentro mediante entradas (entries).
- Los protected objects garantizan que solo una tarea a la vez pueda ejecutar un procedimiento protegido, sustituyendo de forma segura los conceptos tradicionales de semáforos y mutexes.
- Ada incluye un planificador de tareas integrado que asigna tiempo de CPU a las distintas tareas en ejecución.

Los cambios que hice fueron: `real` → `Float`, sustituir los "bloques try-catch" por `begin-exception-when` (con el ejemplo de sintaxis correcta), aclarar que colas/árboles/conjuntos/mapas no son nativos, y reemplazar semáforos y mutexes por protected objects y rendezvous, que son los mecanismos reales de Ada.

Ejercicio 6 - Lenguaje JAVA

Diga para qué fue, básicamente, creado Java. ¿Qué cambios le introdujo a la Web? ¿Java es un
lenguaje dependiente de la plataforma en dónde se ejecuta? ¿Por qué?

¿Para qué fue creado Java?

Java fue creado en 1991 por James Gosling y su equipo en Sun Microsystems, originalmente bajo el nombre *Oak*. El objetivo inicial era desarrollar un lenguaje para dispositivos electrónicos embebidos como televisores y electrodomésticos, donde el software debía poder ejecutarse en hardware muy variado. De ahí surgió su principio fundamental: **"write once, run anywhere"** — escribir el código una sola vez y ejecutarlo en cualquier plataforma sin modificaciones.

¿Qué cambios le introdujo a la Web?

Cuando Internet comenzó a crecer masivamente a mediados de los 90, Java se adaptó al contexto web y produjo cambios importantes. Permitió que las páginas web dejaran de ser estáticas y pudieran incluir contenido interactivo mediante los **applets**, pequeños programas que se ejecutaban directamente en el navegador. También introdujo la posibilidad de ejecutar lógica del lado del servidor mediante **servlets**, sentando las bases de las aplicaciones web dinámicas. En términos más amplios, Java popularizó la idea de aplicaciones distribuidas y cliente-servidor sobre Internet.

¿Java es dependiente de la plataforma?

No, Java es **independiente de la plataforma**, y esto es una de sus características más importantes. El motivo es su arquitectura de dos pasos:

1. El código fuente `.java` se compila a **bytecode** (archivos `.class`), que no es código máquina nativo sino un formato intermedio.
2. Ese bytecode es ejecutado por la **Java Virtual Machine (JVM)**, que sí es específica de cada plataforma.

La JVM actúa como capa intermedia: existe una versión de JVM para Windows, otra para Linux, otra para Mac, etc. El bytecode es siempre el mismo — lo que cambia es la JVM que lo interpreta. Así el programador escribe y compila una sola vez, y el programa corre en cualquier sistema que tenga una JVM instalada.

Ejercicio 7 - Lenguaje JAVA

Java está basado principalmente en **C y C++**, de los cuales hereda gran parte de su sintaxis: las llaves para delimitar bloques, los tipos de datos básicos, los operadores y las estructuras de control como `if`, `for` y `while`. Sin embargo, eliminó o modificó las partes más peligrosas de C++ como la aritmética de punteros, la herencia múltiple de clases y la gestión manual de memoria, reemplazando esta última por un **garbage collector** automático.

También tomó influencias de **Smalltalk** en cuanto al modelo de objetos y la idea de ejecutar el código sobre una máquina virtual, y de **Objective-C** en algunos aspectos del manejo de interfaces. En resumen, Java puede verse como un C++ simplificado, más seguro y orientado completamente a objetos, diseñado para ser robusto y portable.

Ejercicio 8 - Lenguaje JAVA

¿Qué son los Applets?

Los applets son pequeños programas Java diseñados para ejecutarse dentro de un navegador web. Se incrustaban en páginas HTML y se descargaban automáticamente al cliente cuando este visitaba la página. Se ejecutaban dentro de la JVM del navegador en un entorno restringido llamado **sandbox**, que limitaba lo que el applet podía hacer por razones de seguridad — por ejemplo, no podía acceder al sistema de archivos del usuario sin permiso explícito.

Fueron muy populares durante los años 90 y principios de los 2000 para agregar interactividad a las páginas web, como animaciones, juegos o formularios complejos. Sin embargo, con el tiempo fueron cayendo en desuso por problemas de seguridad, lentitud de carga y la aparición de alternativas como JavaScript y HTML5. En 2017 fueron oficialmente deprecados y en 2019 eliminados del JDK.

¿Qué son los Servlets?

Los servlets son programas Java que se ejecutan del lado del **servidor**, no del cliente. Reciben peticiones HTTP, las procesan y devuelven una respuesta, generalmente una página HTML generada dinámicamente. Son el equivalente servidor de los applets.

A diferencia de los applets, los servlets tuvieron un éxito enorme y siguen siendo relevantes hoy. Son la base sobre la que se construyeron frameworks como **Spring** y **Jakarta EE** (antes Java EE), que dominan el desarrollo de aplicaciones web empresariales. La diferencia clave con los applets es que el usuario nunca ve ni ejecuta el código del servlet — toda la lógica ocurre en el servidor y el cliente solo recibe el resultado.

	Applet	Servlet
Se ejecuta en	Cliente (navegador)	Servidor
Estado actual	Obsoleto	Vigente
Acceso al hardware	Restringido (sandbox)	Completo (en el servidor)
Genera	Interfaz gráfica	Respuestas HTTP

Ejercicio 9 - C

¿Cómo es la estructura de un programa escrito en C? ¿Existe anidamiento de funciones?

Un programa en C tiene una estructura bien definida y bastante rígida comparada con lenguajes modernos. Sus componentes principales son:

```
/* 1. Directivas de preprocesador */
#include <stdio.h>
#include <stdlib.h>
#define PI 3.14159

/* 2. Declaraciones globales (variables y prototipos de fun
```

```
ciones) */
int contador = 0;
float calcularArea(float radio);

/* 3. Función principal — punto de entrada obligatorio */
int main() {
    float r = 5.0;
    float area = calcularArea(r);
    printf("Area: %f\n", area);
    return 0;
}

/* 4. Definición de funciones */
float calcularArea(float radio) {
    return PI * radio * radio;
}
```

Las partes son:

- **Directivas de preprocesador:** líneas que empiezan con `#` y se procesan antes de la compilación. `#include` incorpora bibliotecas y `#define` define constantes o macros.
- **Declaraciones globales:** variables y prototipos de funciones visibles en todo el programa.
- **Función `main`:** es el punto de entrada obligatorio de todo programa C. El sistema operativo la llama al ejecutar el programa. Devuelve un entero (`0` indica ejecución exitosa).
- **Funciones definidas por el usuario:** bloques de código reutilizables definidos fuera del `main`.

¿Existe anidamiento de funciones?

No, C **no permite definir funciones dentro de otras funciones**. Todas las funciones deben estar definidas al mismo nivel (nivel global). Esto es una diferencia importante con lenguajes como Python o JavaScript donde sí es posible definir funciones anidadas.

```
/* Esto es INVÁLIDO en C */
int main() {
```

```
    int suma(int a, int b) {  // ERROR — no se puede
        return a + b;
    }
    return 0;
}

/* Lo correcto en C */
int suma(int a, int b) {  // definida fuera del main
    return a + b;
}

int main() {
    int resultado = suma(3, 4);
    return 0;
}
```

Lo que sí existe es el **anidamiento de llamadas** a funciones (que una función llame a otra), pero no la definición de una función dentro de otra.

Ejercicio 10 - Lenguaje C

Describa el manejo de expresiones que brinda el lenguaje.

C es un lenguaje muy rico en tipos de expresiones. Se destaca por permitir expresiones compactas y potentes, aunque a veces difíciles de leer.

Expresiones aritméticas

Operadores clásicos, con una particularidad: la división entre enteros trunca el resultado.

```
int a = 10, b = 3;
int cociente = a / b;    // 3 (no 3.33)
int resto    = a % b;    // 1 (módulo)
float div    = 10.0 / 3; // 3.333 (al menos uno debe ser fl
oat)
```

Expresiones relacionales

Devuelven `1` (verdadero) o `0` (falso), ya que C no tiene un tipo booleano nativo y usa enteros para representar verdad o falsedad.

```
int x = 5;
x > 3    // 1 (verdadero)
x == 10  // 0 (falso)
x != 3   // 1 (verdadero)
```

Expresiones lógicas

```
int edad = 20, tiene_dni = 1;
if (edad >= 18 && tiene_dni) { ... }  // AND
if (edad < 18 || !tiene_dni) { ... }  // OR, NOT
```

Expresiones de asignación

Una característica distintiva de C es que la asignación es una **expresión**, no una sentencia, ya que devuelve el valor asignado. Esto permite asignaciones encadenadas y también usarlas dentro de condiciones.

```
int a, b, c;
a = b = c = 10;       // asignación encadenada, los tres va
len 10

// asignaciones compuestas
a += 5;   // a = a + 5
a -= 3;   // a = a - 3
a *= 2;   // a = a * 2
a /= 4;   // a = a / 4
a %= 3;   // a = a % 3
```

Expresiones de incremento y decremento

C incorpora los operadores `++` y `--`, que pueden usarse en forma prefija o sufija, con comportamientos distintos:

```
int x = 5;
int a = x++;  // a = 5, luego x = 6 (sufijo: usa el valor,
después incrementa)
int b = ++x;  // x = 7, luego b = 7 (prefijo: incrementa pr
imero, después usa)
```

Expresión condicional (operador ternario)

```
int max = (a > b) ? a : b;  // devuelve a si a > b, sino b
```

Expresiones con punteros

Una de las características más potentes y peligrosas de C. Los operadores `&` (dirección) y `*` (desreferencia) permiten trabajar directamente con memoria.

```
int x = 10;
int *p = &x;   // p guarda la dirección de x
*p = 20;       // modifica x a través del puntero, ahora x
= 20
```

Expresiones de manejo de bits

C permite operar directamente sobre los bits de un valor, algo muy útil en programación de sistemas y hardware.

```
int a = 0b1100;  // 12 en binario
int b = 0b1010;  // 10 en binario

a & b   // AND bit a bit: 0b1000 = 8
a | b   // OR  bit a bit: 0b1110 = 14
a ^ b   // XOR bit a bit: 0b0110 = 6
~a      // NOT bit a bit
a << 2  // desplazamiento izquierda: multiplica por 4
a >> 1  // desplazamiento derecha:  divide por 2
```

Expresiones de casting

C permite convertir explícitamente un tipo en otro mediante el cast. También ocurren conversiones implícitas cuando se mezclan tipos en una misma expresión.

```
int a = 5;
float resultado = (float) a / 2;  // cast explícito → 2.5
                                   // sin cast sería 2 (div
isión entera)
```

En resumen, C ofrece un conjunto de expresiones muy completo y eficiente, pero exige entender bien su funcionamiento interno, especialmente en casos como la división entera, la diferencia entre `++` prefijo y sufijo, y el manejo de punteros, donde un error puede tener consecuencias difíciles de detectar.

Ejercicio 11 - Lenguaje Python, Ruby, Php

Python *¿Qué programas se pueden escribir?*
Python es un lenguaje de propósito general. Se usa para desarrollo web (Django, Flask), ciencia de datos e inteligencia artificial (pandas, NumPy, TensorFlow), automatización de tareas, scripting, aplicaciones de escritorio y scripting del sistema operativo. Es el lenguaje dominante en machine learning y data science.

Paradigma: Python es **multiparadigma** — soporta programación imperativa, orientada a objetos y funcional. El programador elige el estilo según el problema.

Características que determinan la pertenencia:

- OOP: soporta clases, herencia, polimorfismo y encapsulamiento
- Funcional: funciones de primera clase, `map`, `filter`, `lambda`, comprensiones de listas
- Imperativo: estructuras de control clásicas (`if`, `for`, `while`)

Ruby *¿Qué programas se pueden escribir?*
Ruby es especialmente popular para desarrollo web gracias a su framework **Ruby on Rails**, que revolucionó la forma de construir aplicaciones web en los 2000. También se usa para scripting, automatización y herramientas de línea de comandos.

Paradigma: **Orientado a objetos puro**, con soporte funcional. En Ruby absolutamente todo es un objeto, incluso los números y los booleanos.

Características que determinan la pertenencia:

- Todo es un objeto: `5.times { puts "hola" }` — el número 5 recibe un mensaje
- Bloques, procs y lambdas para programación funcional
- Mixins mediante módulos como alternativa a la herencia múltiple
- Énfasis en la expresividad y en que el código sea "feliz de leer"

PHP *¿Qué programas se pueden escribir?*
PHP fue diseñado específicamente para **desarrollo web del lado del servidor**. Genera HTML dinámico, interactúa con bases de datos y maneja sesiones y formularios. Potencia sitios como WordPress, Facebook (en sus inicios) y Wikipedia. Fuera del contexto web tiene muy poco uso.

Paradigma: **Multiparadigma**, originalmente imperativo/procedural, con soporte de OOP agregado desde PHP 5. No es tan orientado a objetos puro como Ruby ni tan funcional como Python.

Características que determinan la pertenencia:

- Procedural: funciones globales, variables y estructuras de control clásicas
- OOP: clases, interfaces, herencia, traits (similar a mixins)
- Se integra directamente con HTML mediante etiquetas `<?php ?>`

Ejercicio 12 - Lenguaje Python, Ruby, Php

Cite otras características importantes de Python, Ruby, PHP, Gobstone y Processing.
Por ejemplo: tipado de datos, cómo se organizan los programas, etc

Python

- *Tipado:* dinámico y fuerte. Las variables no se declaran con tipo, pero una vez que tienen un valor, no se hacen conversiones implícitas entre tipos incompatibles (`"5" + 3` lanza error, a diferencia de JS).
- *Organización del programa:* mediante funciones, clases y módulos. Cada archivo `.py` es un módulo importable. Los paquetes son carpetas con un archivo `__init__.py`.
- *Indentación obligatoria:* los bloques de código se delimitan por indentación, no por llaves. Esto fuerza la legibilidad.
- *Garbage collector:* manejo automático de memoria mediante conteo de referencias.
- *Comprensiones de listas:* forma concisa de crear listas.

```
cuadrados = [x**2 for x in range(10)]
```

- *Duck typing:* si un objeto tiene el método esperado, puede usarse sin importar su tipo formal.

Ruby

- *Tipado:* dinámico y fuerte, similar a Python.
- *Organización del programa:* mediante clases, módulos y mixins. Los módulos cumplen dos funciones: namespacing y mixins (se incluyen en clases para agregar comportamiento sin herencia).
- *Todo es un objeto:* no existen tipos primitivos. Incluso `nil` (equivalente a null) es un objeto.
- *Bloques y yielding:* los métodos pueden recibir bloques de código y ejecutarlos internamente con `yield`, lo que hace al lenguaje muy expresivo.

```
[1, 2, 3].each { |n| puts n * 2 }
```

- *Sintaxis flexible:* los paréntesis en las llamadas a métodos son opcionales, los métodos que devuelven booleano terminan en `?`, los que modifican el objeto terminan en `!`.
- *Open classes:* las clases existentes, incluso las del lenguaje, pueden ser modificadas en tiempo de ejecución (monkey patching).

PHP

- *Tipado:* dinámico y débil. Hace coerciones implícitas de tipos, lo que puede generar comportamientos inesperados similares a JS.
- *Organización del programa:* procedural con funciones globales, o mediante clases y namespaces desde PHP 5. Los archivos se incluyen con `include` o `require`.
- *Integración con HTML:* el código PHP se mezcla directamente con HTML, lo que facilita la generación dinámica de páginas pero puede volverse difícil de mantener.

```
<h1>Hola, <?php echo $nombre; ?></h1>
```

- *Traits:* mecanismo para reutilizar código entre clases sin herencia, similar a los mixins de Ruby.
- Superglobales:* variables como `GET‘,‘_POST`, `SESSION‘y‘_COOKIE` están disponibles en cualquier parte del programa y facilitan el manejo de datos

web.

GET‘,‘_GET`, `

SESSION‘y‘_SESSION` y `

- *Manejo de bases de datos:* tiene integración nativa con MySQL y otras bases mediante PDO, lo que lo hace muy conveniente para aplicaciones web con persistencia de datos.

Gobstones

- *Propósito:* es un lenguaje diseñado **exclusivamente para enseñar programación** a principiantes, creado en Argentina por la Universidad Nacional de Quilmes.
- *Paradigma:* imperativo y procedural. No tiene OOP.
- *Tipado:* simple, con tipos básicos como números, booleanos, colores y direcciones.
- *Organización:* mediante procedimientos (que ejecutan acciones) y funciones (que devuelven valores). El programa principal es el procedimiento `program`.
- *Entorno:* opera sobre un tablero de celdas donde se pueden poner y sacar bolitas de colores, lo que hace la ejecución visual e intuitiva para aprender conceptos como iteración y condición.

```
program {
    Poner(Rojo)
    Mover(Norte)
    Poner(Azul)
}
```

Processing

- *Propósito:* lenguaje y entorno diseñado para **arte generativo, visualización de datos y prototipado visual**. Muy usado por artistas y diseñadores que quieren programar.
- *Paradigma:* imperativo, basado en Java (es esencialmente Java simplificado con bibliotecas gráficas integradas).
- *Tipado:* estático, hereda el sistema de tipos de Java.

- *Organización:* los programas se estructuran en torno a dos funciones principales: `setup()` que se ejecuta una vez al inicio, y `draw()` que se ejecuta en bucle como los frames de una animación.

```
void setup() {
    size(400, 400);  // tamaño del lienzo
}

void draw() {
    background(255);
    ellipse(mouseX, mouseY, 50, 50);  // círculo que sigue
el mouse
}
```

- *Biblioteca gráfica integrada:* tiene funciones nativas para dibujar formas, manejar colores, reproducir sonido y capturar video, sin necesidad de bibliotecas externas.
- *Comunidad:* muy activo en educación artística y visualización científica.

Ejercicio 13 - Javascript

¿A qué paradigma corresponde JavaScript?

JavaScript es un lenguaje **multiparadigma**, lo que significa que no impone un único estilo de programación sino que permite combinar varios según la necesidad:

Paradigma imperativo/procedural: se pueden escribir programas como una secuencia de instrucciones con estructuras de control clásicas (`if`, `for`, `while`), sin necesidad de usar clases ni funciones de orden superior.

Paradigma orientado a objetos: JavaScript soporta OOP, aunque con una particularidad histórica importante — su modelo original está basado en **prototipos** y no en clases. Cada objeto puede heredar directamente de otro objeto (prototipo), sin necesidad de definir una clase primero. Desde ES6 (2015) se agregó la sintaxis de `class` que resulta más familiar, pero por debajo sigue siendo herencia prototípica.

```
// OOP con clases (ES6+)
class Animal {
    constructor(nombre) {
```

```
        this.nombre = nombre;
    }
    hablar() {
        return `${this.nombre} hace un sonido`;
    }
}
```

Paradigma funcional: JavaScript trata las funciones como **ciudadanos de primera clase** — pueden asignarse a variables, pasarse como argumento y devolverse como resultado. Esto habilita programación funcional con `map`, `filter`, `reduce`, closures y funciones de orden superior.

```
const doble = x => x * 2;
const resultado = [1, 2, 3].map(doble); // [2, 4, 6]
```

¿A qué tipo de lenguaje pertenece?

JavaScript es un lenguaje:

- **Interpretado / JIT compilado:** originalmente era puramente interpretado, pero los motores modernos como V8 (Chrome y Node.js) usan compilación Just-In-Time que convierte el código a código máquina en tiempo de ejecución para mejorar la performance.
- **De alto nivel:** abstrae completamente el manejo de memoria y los detalles del hardware.
- **De propósito general:** aunque nació para el navegador, hoy con Node.js se usa en servidores, aplicaciones de escritorio, dispositivos IoT y más.
- **De scripting:** originalmente diseñado para escribir scripts cortos que se ejecutan en el navegador sin paso de compilación explícito.

Ejercicio 13 - Javascript

Tipado de datos

JavaScript tiene tipado **dinámico y débil**:

- *Dinámico:* las variables no tienen tipo fijo — el tipo lo tiene el valor, no la variable. Una misma variable puede contener un número, luego un string, luego un objeto.

- *Débil:* realiza coerciones implícitas de tipos que pueden generar resultados sorprendentes.

```
// Tipado dinámico
let x = 10;
x = "hola";   // válido, ahora x es string
x = true;     // válido, ahora x es booleano

// Coerción implícita (tipado débil)
"5" + 3       // "53" (convierte el número a string)
"5" - 3       // 2   (convierte el string a número)
true + 1      // 2   (true se convierte a 1)
null + 1      // 1   (null se convierte a 0)
```

Los tipos de datos en JavaScript son:

- *Primitivos:* `Number`, `String`, `Boolean`, `undefined`, `null`, `Symbol`, `BigInt`
- *Por referencia:* `Object`, `Array`, `Function`

Una particularidad es que `typeof null` devuelve `"object"`, lo cual es un bug histórico del lenguaje que se mantuvo por compatibilidad.

Variables

JavaScript tiene tres formas de declarar variables, con comportamientos distintos:

```
var x = 10;   // alcance de función, tiene hoisting, evitar
en código moderno
let y = 20;   // alcance de bloque, sin hoisting, reasignab
le
const z = 30; // alcance de bloque, sin hoisting, no reasig
nable
```

- `var` tiene **hoisting**: la declaración se mueve al inicio de la función en tiempo de ejecución, lo que puede generar comportamientos confusos.
- `let` y `const` fueron introducidos en ES6 y son los recomendados actualmente.

- `const` no significa que el valor sea inmutable — un objeto declarado con `const` puede tener sus propiedades modificadas, solo que la variable no puede apuntar a otro objeto.

```
const persona = { nombre: "Ana" };
persona.nombre = "Juan";  // válido
persona = {};             // ERROR — no se puede reasignar
```

Manejo de excepciones

JavaScript usa bloques `try-catch-finally` para el manejo de errores:

```
try {
    let resultado = dividir(10, 0);
} catch (error) {
    console.log("Error:", error.message);
} finally {
    console.log("Esto se ejecuta siempre");
}
```

Se pueden lanzar excepciones con `throw`, y se pueden crear excepciones personalizadas extendiendo la clase `Error`:

```
class DivisionPorCeroError extends Error {
    constructor() {
        super("No se puede dividir por cero");
        this.name = "DivisionPorCeroError";
    }
}

function dividir(a, b) {
    if (b === 0) throw new DivisionPorCeroError();
    return a / b;
}
```

Scope y closures

El scope define dónde es accesible una variable. JavaScript tiene scope de función (con `var`) y scope de bloque (con `let` y `const`). Un **closure** ocurre

cuando una función interna recuerda las variables de su función contenedora aunque esta ya haya terminado de ejecutarse:

```
function contador() {
    let count = 0;
    return function() {
        count++;
        return count;
    };
}

const incrementar = contador();
incrementar(); // 1
incrementar(); // 2
incrementar(); // 3
```

Asincronismo

Una característica fundamental de JavaScript es su modelo **asincrónico y no bloqueante**, basado en un event loop de un solo hilo. Evolucionó en tres etapas:

```
// 1. Callbacks (original, genera "callback hell")
fetch(url, function(data) {
    procesar(data, function(resultado) { ... });
});

// 2. Promesas (ES6)
fetch(url)
    .then(data => procesar(data))
    .then(resultado => console.log(resultado))
    .catch(error => console.error(error));

// 3. Async/Await (ES8 — la forma moderna)
async function obtenerDatos() {
    try {
        const data = await fetch(url);
        const resultado = await procesar(data);
        console.log(resultado);
```

```
    } catch (error) {
        console.error(error);
    }
}
```

Prototipos

Antes de ES6, la herencia en JavaScript se implementaba directamente mediante la cadena de prototipos. Todo objeto tiene una referencia a su prototipo, y si se busca una propiedad que no existe en el objeto, JavaScript la busca en el prototipo, luego en el prototipo del prototipo, y así sucesivamente:

```
const animal = {
    hablar() { return "..."; }
};

const perro = Object.create(animal); // perro hereda de ani
mal
perro.nombre = "Rex";
perro.hablar(); // encuentra el método en el prototipo
```

En resumen, JavaScript es un lenguaje extremadamente flexible y ubicuo, pero esa flexibilidad tiene un costo: requiere que el programador conozca bien sus particularidades para evitar bugs difíciles de detectar, especialmente los relacionados con la coerción de tipos, el comportamiento de `this` y el asincronismo.